Book

1) What is a Database?

Definition 1 (Database (DB))**.**

A database is a permanent collection of related data, where data indicates known facts that can be recorded and have intrinsic meaning

A database represents a **mini universe** or Universe of Discourse (UoD) which is it's own mini-universe with a well defined set of rules, it is the *domain* in which we will work. This rules allow for a coherent data collection which should also be adjusted for the needs of users/shareholders.

Definition 2 (Database Management System (DBMS))**.**

A database management system (DBMS) is a general purpose software system which allows users to create, manage, and update a database

A DBMS must do the following:

- Define the DB: the **schema** contains data types, structure, constraints) stored in system catalog as metadata
- Construct the DB: correctly store the data
- Manipulate the DB: query and update data correctly

1.1) Features of a DB

What is the difference between a DB and an archive?
The core features of a DB are the following:

- Size: databases are big
- Sharing: DB are shared among applications and users. This must account into **redundancy and inconsistency**. Moreover, there must be **concurrency control** (isolation) for undesired actions between users/apps and also **atomicity** to avoid partial/incorrect operations o the data
- Durability of DBMS: they must be able to operate for long times with no data losses also in case of hw/sw malfunctioning
- Security of DBMS: users and applications can access, read and write only upon authentication and authorization
- Efficiency of DBMS: optimization in time and space

A **file system is not a db.** It allows for big amount of data but has a limited schema (tree) and doesn't guarantee efficiency, durability, redundancy and consistency etc.

We divide the architecture in 3 schemas:

- **External:** describes relevant data for each application
- **Logical:** integrated representation of the data (independent from phy)
- **Internal:** physiscal representation as data structure and storage units

Tis allows for 2 levesl of data independence:

- **Physical idependence:** external and logical can remain fixed while phy changes
- **Logical independence:** external level independent from logical one

2) Database Design

To develop a software we must follow these steps:
Feasibility Study → Requirement Analysis → Design(data+app) → Development → Testing → Operation and Maintenance.

- Feasibility Study: check if and how the aim of the software is achievable and how much does it cost based on the resources and time available
- **Requirement analysis:** define what kind of data and how it is stored/recorded
- Design: data and application design; this is the actual modeling part
- Development:
- Testing:
- Operation and Maintenance: no software runs without maintenance, this impacts costs and feasibility

These steps are iterative, the more you work on the project, the more you review the various studies.

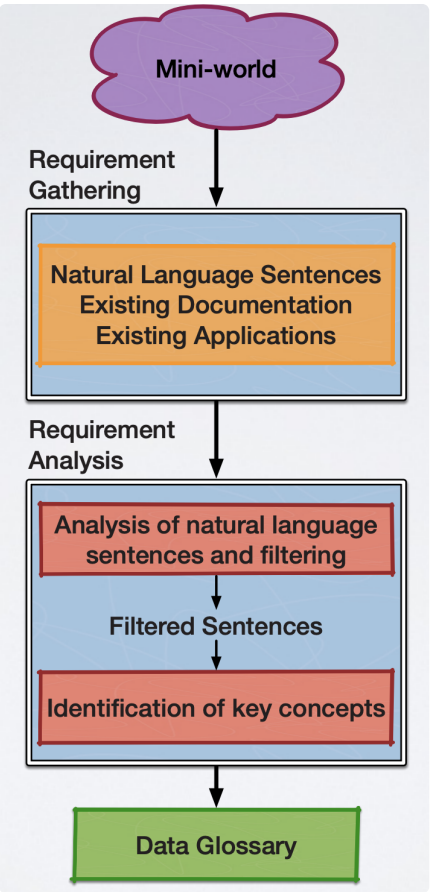
2.1) Requirement Analysis

Definition 3 (Requirements)**.**

Requirements are the set of features that a system must have to comply with its purpose

There are different kinds of requirements:

- Functional Requirements: what the system must do
- Non-functional Requirements: the way in which the system must do
- Other Constraints: general requirements by stakeholder often defined ahead



Requirements Gathering and Analysis

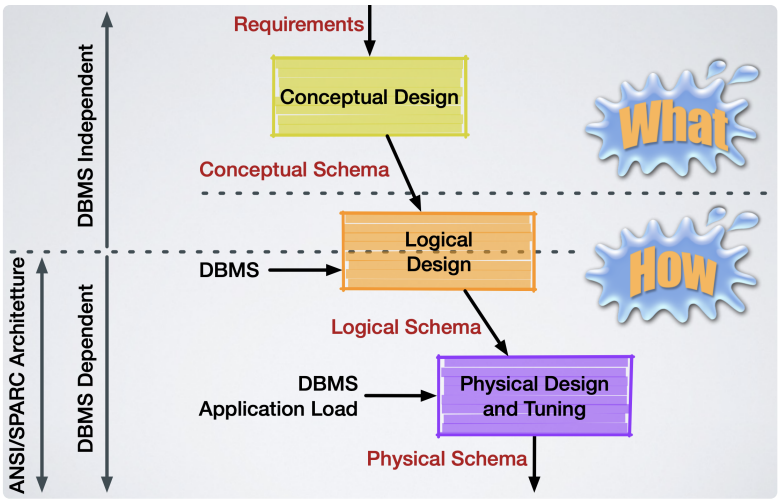
Software must be adapted to many different **user**. Once the main group is found there will be subgroups (admin, superuser, common user, guest, etc.). All of them have different requirements that need to be held account to and modeled correctly.

Errors must also be handled, both in production and in development.

2.2) Database Design

Definition 4 (Goals of the DB Design)**.**

The database design aims at defining the logical schema and the physical schema (see the ANSI/ SPARC architecture) of a database, according to the outcomes of the requirement analysis



Design Schema

Definition 5 (Data Model)**.**

A (data) model is a set of symbolic structures used to describe the representation of a mini-world of interest. This representation is called schema

The schema is the structure of the DB and is the **intensional** level of the DB. The specific **instance** is the actual data that represents the **extensional** level of the DB

The quality of a schema is based on the following properties:

- Completeness: a schema must represent all the concepts and their properties at a mini-world relevant level with the correct identified requirements
- Correctness: the structures must be used properly and accordingly to the described semantics
- Minimality: each concept is represented only once (duplicates may still exist but must be motivated and documented)
- Readability: the schema should be easy to read and self explanatory

Definition 6 (Conceptual Design)**.**

Representation of the mini-world by means of a high-level formal model, integrating all the relevant concepts and independent from the DBMS

This schema is used to be understandable by everyone (stakeholders) in order to reduce the risk of misunderstanding. It focuses on the abstraction of the mini-world

Input: description of mini world from requirement analysis:
Output: conceptual schema + constraints

Definition 7 (Logical Design)**.**

Representation of the mini-world by means of logical structures, independent from physical structures and characteristic of a class of DBMS

Input: conceptual schema, class of DBMS, app load
Output: logical schema + constraints

Definition 8 (Physical Design)**.**

Representation of the mini-world by means of physical data structures specific to a given DBMS

Input: logical schema
Output: physical schema and tuning on DBMS

3) Entity-Relationship Model

The Entity-Relationship (ER) model provides several constructs that impact on both the intensional level (schema) and the extensional level (instance). Tis step allows us to represent the conceptual schema (requirements analysis) in a graphical form

3.1) Entity

Definition 9 (Entity)**.**

An entity is a class of objects of the real world (facts, people, things) which exist autonomously and have shared properties

Each entity has an **unique name** that univocally identifies it.

At the **extensional** level an entity is a set of objects called instances. In a schema S in which an entity e is defined, then in each instance i of the schema S, the entity e is associated with a set of objects, also called the **extension** of e in the instance i of S.

$$\begin{aligned}\text{instance: } & I \times S \to I \\ & (i, e) \to \text{instance}(i, e) = \{e_1, e_2, \dots, e_n\}\end{aligned}$$

The instance itself e_i is not a value that identifies the object, but it is the object itself.

3.2) Relationship

Definition 10 (Relationship)**.**

A relationship represents an association among two or more entities. The number of entities participating in a relationship is called the degree of the relationship

Each relationship has an **unique name** that univocally identifies it

At **extensional** level we have:

$$\begin{aligned}\text{instance: } & I \times S \to I \times I \\ & (i, r) \to \text{instance}(i, r) \subseteq \text{instance}(i, e) \times \text{instance}(i, f) = \{(e_1, f_1), \dots, (e_n, f_n)\}\end{aligned}$$

Each relationship must be paired with entities, you cannot do a relationship that concatenates with another relationship.

Relationships are usually **binary**, that is, we link two entities, but it can also be **N-ary** where N entities are linked.

Moreover relationships can also be **recursive**, however to do so we must also define distinct roles, otherwise the relationships don't hold.

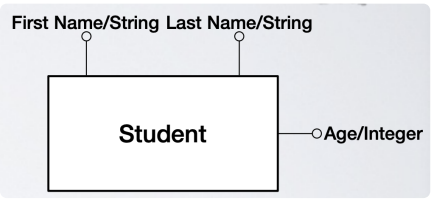
$$r_k = (u_1 : e_{1,i}, u_2 : e_{2,j}, \dots, u_n : e_{n,h})$$

3.3) Attribute

Definition 11 (Entity Attribute)**.**

An attribute of an entity is a local property of that entity, relevant for the application. An attribute maps each instance of an entity to a value belonging to a set called domain

Each Attribute has a name which univocally identifies it in the entity. It is represented by a circle connected to the entity. The domain of the attribute is the universe of which it comes from (int, string, float, bool and subset of those)



Example

At an extensional level we have:

$$\begin{aligned}\text{instance}: & I \times S \to I \times D \\ & (i,a) \to \text{instance}(i,a) \subseteq \text{instance}(i,e) \times D = \{(e_1,v_1),\dots,(e_n,v_n)\} \\ & \forall e_i \in \text{instance}(i,e) \exists! v_j \in D | (e_i,v_j) \in \text{instance}(i,a)\end{aligned}$$

Therefore the attribute has

$$\begin{aligned}a: & \text{instance}(i,e) \to D \\ & e_i \to v_j\end{aligned}$$

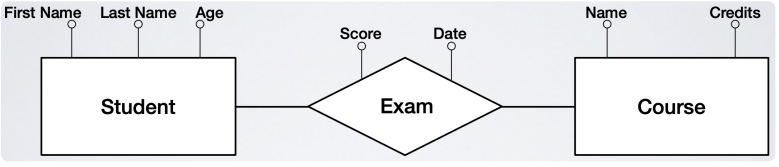
Sometimes a specific entity doesn't have the attribute to associate, therefore we can insert the value **NULL** as the attribute value, it can mean

- Value undefined: an appropriate value doesn't exist (a degree attribute doesn't apply for those who didn't attend uni)
- Value not available: an appropriate value exists but it is not know (person doesn't disclose it's age)
- Value unknown: an appropriate value may or may not exist (a person may or may not have a phone number)

Definition 12 (Relationship Attribute)**.**

An attribute of a relationship is a local property of that relation, relevant for the application. An attribute maps each instance of a relationship to a value belonging to a set called domain

It is the same as before but notice that a relationship attribute isn't a property of an entity participating in the relationship, but a property of the association between entities



Example

The extensional level is very similar:

$$\text{instance}: I \times S \to I \times D$$

$$(i, a) \to \text{instance}(i, a) \subseteq \text{instance}(i, r) \times D = \{(r_1, v_1), \ldots, (r_n, v_n)\} = \{((e_1, f_1), v_1), .., ((e_n, f_n), v_n)\}$$
$$\forall e_i \in \text{instance}(i, e) \exists! v_j \in D | (e_i, v_j) \in \text{instance}(i, a)$$

Finally also attributes can have cardinalities.

3.4) Integrity Constraints

Definition 13 (Integrity Constraint)**.**

An integrity constraint is a rule expressed on the schema (intensional) that specifies a condition which must be met for each instance (extensional) of the schema

There are mainly 4 types of constraints:

- Cardinality constraints on relationships
- Cardinality constraints on attributes
- Identification constraints on entities
- Other constraints

Definition 14 (Cardinality Constraints on Relationships)**.**

A cardinality constraint refers to a role (u) of an entity (e) in a relationship (r) and it expresses a lower an upper bound to the number of instances of the relationship (r) to which each instance of the entity (e) can take part with the role (u)

The definition is a bit complex, in easier words it means that each entity can have constraints on how many relationships it can have with another entity

There are 3 relevant cardinalities:

- **0**: this is an optional participation (minimum) (an entity can have this relationship)
- **1**: this is a mandatory participation (minimum) (an entity must have this relationship) or it can partecipate maximum once in the relationship (maximum) (this relationship is used only once)
- **N**: it can participate many times (maximum)

In particular the constraints:

- $(1, x)$ means that the entity must have the relationship to be valid (an employee is such if it works in a company), **an entity is such if**
- $(1, 1)$ means that the entity must have exactly one type of such relationship (a person is born exactly once)
- (a, b) where $a \leq b$
- A N-ary relationship might have a $(0, x)$ cardinality. This implies that this entity exists only if all the other participants exist.

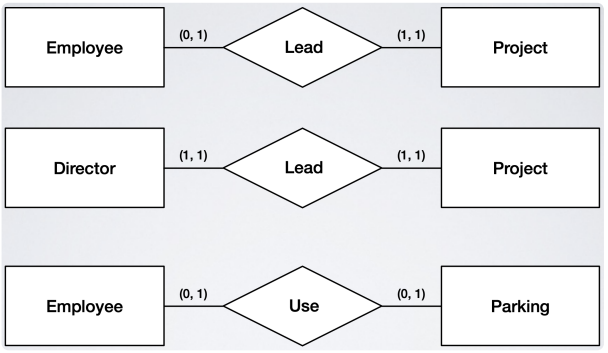
Remark.

A weak entity always has a mandatory participation with its strong entity

In a ternary relation participation is not mandatory, but the instance is created only if all 3 exist

From here we can define 3 classes of relationships

- **One-to-One relationship**: each instance of an entity os associated with one and only one instance of another entity $(0/1, 1) \rightarrow (0/1, 1)$.

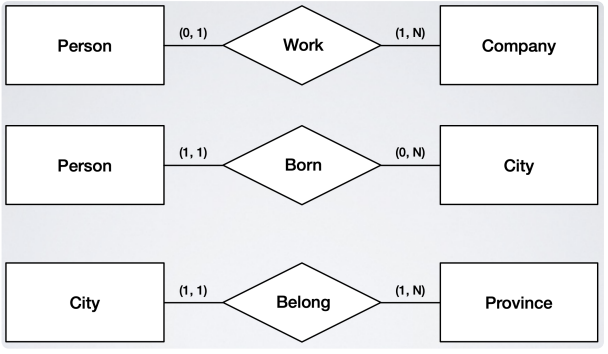


Examples

We can read them as:

1. it is possible for an employee to lead at maximum one project, each project is lead by exactly one employee
2. To be a director it is necessary to lead exactly one project, each project is lead by exactly one director
3. An employee might use one parking spot, a parking spot can be used by one employee or be empty

- **One-to-Many relationship**: each instance of an entity is associated with one or more instances of another entity $(0/1, 1) \rightarrow (0/1, N)$.

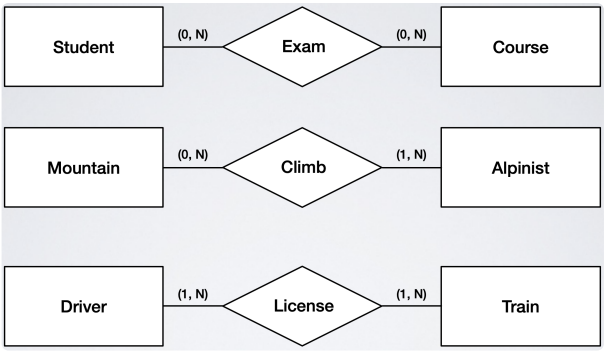


Example

We can read them as:

1. A person might work at one company, and a company can have at least one employee
2. A person is born exactly once in a city, however a city might have no or many people born in it
3. A city belongs to exactly one province, but a province must have at least one city or many (no city in a province doesn't make sense)

- **Many-to-Many relationship**: one or more instances of an entity are associated with one or more instances of another entity

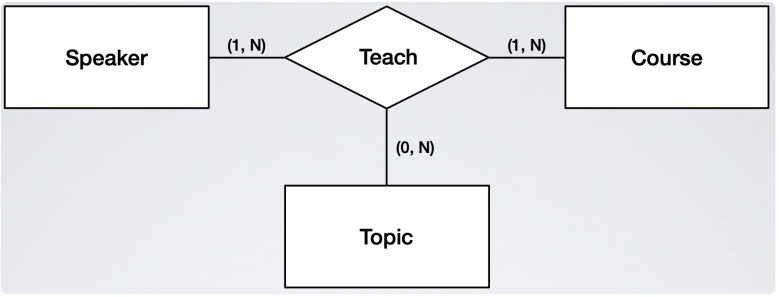


Example

We can read them as:

1. A student can take 0 or many times the exam of a course, a course exam can be taken by none or many students
2. A mountain can be climbed by 0 or many alpinis, but an alpinist must have climbed at least one mountain or many (an alpinist that never climbed a mountain doesn't make sense)
3. A train driver must have one or many licenses (without license he isn't a driver) and a train must be driven by one or many drivers (a train with no driver doesn't make sense)

This can be easily extended to N-ary relationships, however optional relationships must be handled carefully

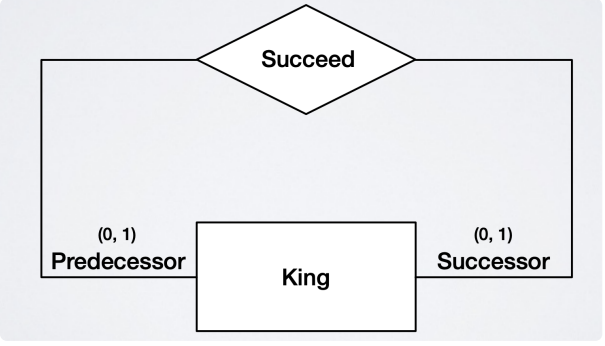


Example

In this example Topic is an optional entity. we can instantiate topics without instantiating course and speaker entities. But the other two entities must be instantiated with also a topic.
We can read this relationship as:

A speaker is such if it teaches one or may courses on one or more topics, a course is such if it is taught by at least a speaker on at least a topic. HOWEVER a topic can exist even if it sn't taught, or it can be taught by one or many courses and by one or many speakers

What about recursions?



Example

In this example it is clear that a King might not have a successor or predecessor, but if it has them, it has just one of them.

$$k_0 \to k_1 \to \ldots \to k_{n-1} \to k_n$$

If the constraints were $(1,1)$ it means that each king has exactly one successor and predecessor, but it is clear that the first king doesn't have a predecessor and the last (current) king doesn't have a successor (yet).

However a problem arises: how do we avoid to set the first king successor of the last king? The model clearly allows for that even though it is wrong. we use identification constraints on entities.

3.5) Identification Constraints on Entities

Definition 15 (Identification Constraint)**.**

An identification constraint for an entity e defines an identifier for the entity, that is a set of properties (attributes or relationships) which allow us to uniquely identify all the instances of an entity.

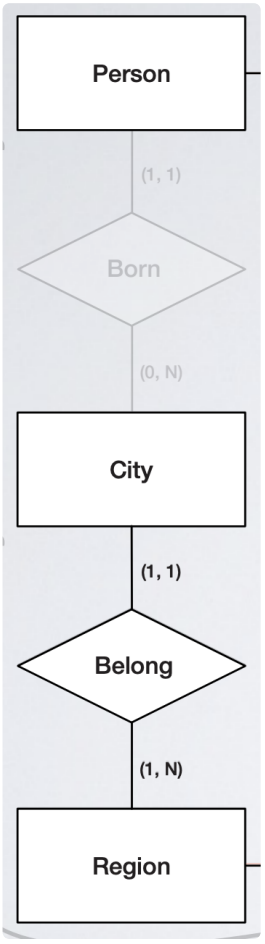
Identifiers can be

- **Internal:** consituted by attributes of the entity with cardinality (1,1), also many attributes can be an identifier
- **External:** by attributes of e (optional identifier itself) and by a relation where the other entity has attributes with identifiers

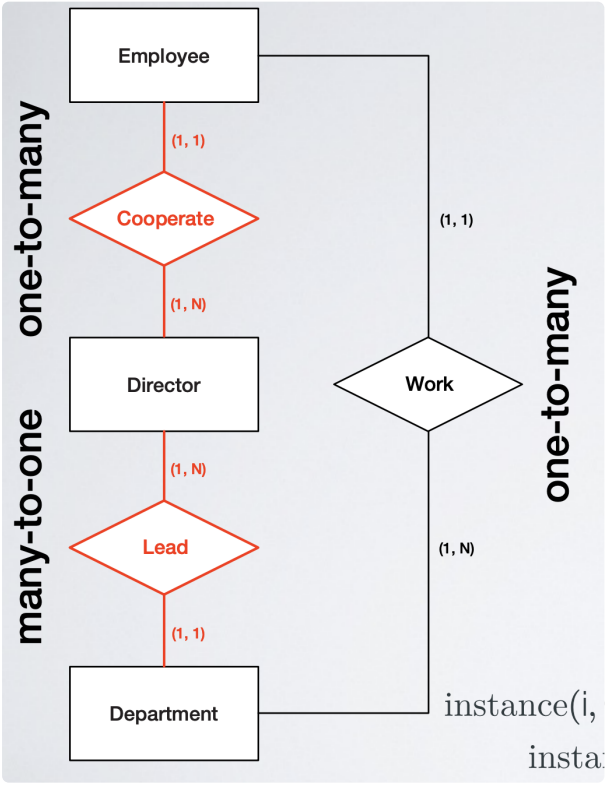
3.6) Redundancy Analysis

It is possible that a chain of relations $R_1, \ldots, R_n$ contains the same information of a single relation R.

Here the external constraint is that a person comes from the region of its birth city.
In this case person and region can be correctly related via a one to many "come from" relation.
However, the "born" relation cannot be removed since it is necessary to know where a person is born.
Also the "Belong" relation cannot be removed



Example of Chain



Example

External constraint: every Employee Cooperates with the Director who Leads the Department where she/he Works.

In this example we can do the following:
-Remove cooperate: still valid and self explanatory
-Remove Lead: still valid but NOT self explanatory
-If we remove work it is worng since a director can lead various departments and therefore the employee isn't linked to the department anymore

Another type of redundancy are derived attributes. Some attributes can be easily calculated on the fly and don't require the insertion of the attribute in the schema.

For example, if we know the birthday we can easily calculate the age of the person when requested.

4) Relational Model

The Relational Model was proposed by E. F. Codd at IBM in 1970 to promote data independence. It is based on the mathematical notion of relation but with differences to let them apply to tables.

Mathematical Relation

Definition 16 (Mathematical Relation)**.**

Given the sets $D_1, \dots, D_n$, the cartesian product is the set of ordered n-uples:

$$D_1 \times \dots \times D_n = \{(d_1, .., d_n) | d_1 \in D_1, \dots d_n \in D_n\}$$

A **relation** (mathematical) is a **subset of the cartesian product:**

$$R \subseteq D_1 \times \dots \times D_n$$

Where D_i are the domain of the relation. A relation over n domains has **degree** n.
The number of n-ples $|R|$ is the cardinality of the relation.
This relations can naturally be represented as a table

Theorem 17 (Properties)**.**

*The relation is a mathematical relation is a set and therefore it is **not ordered** and has **distinct entries***

*Each n-uple in a relation is **ordered** and therefore the i-th value is associated with the i-th domain*

*In a table the **structure is positional**, therefore the role of a domain is disambiguated by its position*

Example:

$$D_1 = (1, 3), D_2 = (x, y) \rightarrow D_1 \times D_2 = \{(1, x), (1, y), (3, x), (3, y)\}$$

Then we can define

$$R_1 = \{(1, x), (3, y)\} \subset D_1 \times D_2$$

In this case $|R| = 2$ and the table is

D1	D2
1	x
3	y

It is impossible, for example to define $R = \{(2, x)\} \notin D_1 \times D_2$
From the properties of the relation we don't know if $(1, x)$ is bigger or smaller than $(3, y)$, however we know that $1 \in D_1$ and $x \in D_2$.

Database Relation

A relation in the relational model is *similar* to the mathematical one, but with some differences:

- components of relation are called **attributes**
- each attribute is characterized by a **name and set of values called domain of attribute**
- the structure is not positional

A relation can be represented as a **table** where **attributes** correspond to **columns** and **attribute names** are used as **column headers**

Definition 18 (Tuple)**.**

Let $X = \{A_1, \dots, A_n\}$ be the set of attributes and $D = \{D_1, \dots, D_n\}$ the set of attribute domains. The following function maps each attribute into its domain:

$$\begin{aligned} \text{dom}: \quad & X \to D \\ & A_i \to D_i \end{aligned}$$

We call tuple over a set of attributes the function which maps each attribute into a value of its domain:

$$\begin{aligned} t_j: \quad & X \to D \\ & A_i \to v_{i,j} \in D_i \end{aligned}$$

This is essentially the entries in the rows. For a tuple t, $\quad t[A_i] \in \text{dom}(A_i) = D_i$

A **relation over a set of attributes** X is the set of tuples over X.

Example:

Home	Visitor	Home Score	Visitor Score
Juventus	Milan	3	1
Lazio	Roma	2	3
Inter	Juventus	3	2
Milan	Lazio	2	0

Example Table

From the mathematical definition we have: $D = \{\text{String}, \text{String}, \text{Int}, \text{Int}\} \longrightarrow R \subset D_1 \times D_2 \times D_3 \times D_4$
Now we can use the database definition to find $A = (\text{Home}, \text{Visitor}, \text{Home Score}, \text{Visitor Score})$
The first tuple is: $t_1 = (\text{Juventus}, \text{Milan}, 3, 1)$

Definition 19 (Relational Schema)**.**

A **relation schema** $R(T)$ consists of a **relation name** R and a **relation type** T denoted by

$$R(A_1 : D_1, \dots, A_n : D_n) \overset{\text{compact}}{\to} R(A_1, .., A_n) \to R(A)$$

where attributes are A_i and attribute domains are D_i

Two relation types are said to be equal if they have same degree (amount of attributes), the same domains and the same attribute names. However **the order of the attributes is irrelevant**

Definition 20 (Relation Instance)**.**

A **relation instace** r of the relation schema $R(A)$, denoted by $r(R)$ is **a set of tuples** $r = \{t_1, \dots, t_m\}$ where each tuple is a labeled list of values

$$t_j = (A_1 : v_{1,j}, \ \dots \ , A_n : v_{n,j})$$

The value of attribute A_i in tuple t_j is indicated as

$$t_j[A_i] = v_{i,j} \in D_i = dom(A_i)$$

And $|r| = m$ is the number of its tuples.

To include **null** value it is possible to extend the domains, that is:

$$t_j[A_i] = v_{i,j} \in D_i \cup \text{NULL}$$

Recall that NULL $\notin D_i$. In a semantic view point NULL is not of the domain, therefore we have absolutely no information.

NULL might have 3 distinct meanings:

- Value undefined: appropriate value doesn't exist for given instance
- Value not available: appropriare value exists, but it is not known
- Value unknown: appropriate value might or might not exist

Now we can finally define the relational schema:

Definition 21 (Database/Relational Schema)**.**

A **database/relational schema** $\mathbf{R}$ consists of a set of of relation schemas $R_i(T_i)$ with different names and denoted by

$$\mathbf{R} = \{R_1(T_1), \dots, R_n(T_n)\}$$

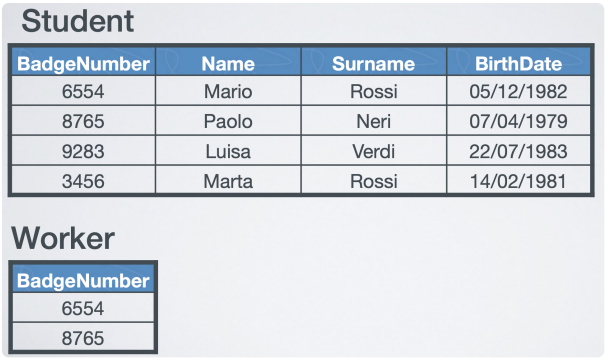
And finally

Definition 22 (Database Instance)**.**

A **database instance** $\mathbf{r}$ of the database schema $\mathbf{R}$ is a set of relation instances

$$\mathbf{r} = \{r_1, \dots, r_n\}$$

Example:



Tables Example

In this example

$$\mathbf{R} = \{\text{Student}(\text{BadgeNumber}, \text{Name}, \text{Surname}, \text{BirthDate}), \text{Worker}(\text{BadgeNumber})\}$$

4.2) Integrity Constraints

Definition 23 (Integrity Constraints in the Relational Model)**.**

An **integrity constraint** os a **condition** expressed on the **database schema** (intensional) and it must be complied with by the **database instances** (extensional) which represents a correct state for the application

It can be a domain constraint, a tuple constraint or a key constraint (intra relational) or a referential integrity constraint (inter relational).

a tuple constraint is a boolean expression that creates conditions of the attributes

Here is a quick rundown of keys

Scenario / Condition	Is it a Super-key?	Is it a Key?	The Rule / Logic
Attributes are Unique	**YES**	**Maybe**	**Uniqueness Rule:** Any set of columns that uniquely identifies a row is a Super-key. It *might* be a Key if it is also minimal.
Attributes are Unique + Minimal	**YES**	**YES**	**Minimality Rule:** If you cannot remove any attribute without losing uniqueness, it is a Key (Candidate Key).
Attributes are Unique + Redundant	**YES**	**NO**	**Non-Minimal Rule:** If you *can* remove an attribute and the remaining set is *still* unique, the original set was just a Super-key, not a Key.
Single Unique Attribute	**YES**	**YES**	**Atomic Rule:** If a single column is unique (e.g., `Badge`), it is automatically a Key because a set of one cannot be reduced further.
Subset of a Key	**NO**	**NO**	**Incomplete Rule:** If you take a piece of a Key (e.g., just `Name` from the composite key), it is usually not unique anymore, so it is neither.

4.3) Mapping

TODO
Weak/Strong entities

ER diagram notation:
Entities in upper case, relation in lower case. Keys have full circle, attributes empty

Relational Model notation:
Both Entities and Relations in upper case. Keys underlined

5) Relational Algebra

We can classify the operations into **update** and **query** operations. Given a database schema and instance:

- an **update** is the function that maps the given database instance into a new database instance which is valid with respect to all the integrity constraints
- a **query** is a function that maps the given database instance into a relation

The 3 main function are an **insertion, update and deletion**

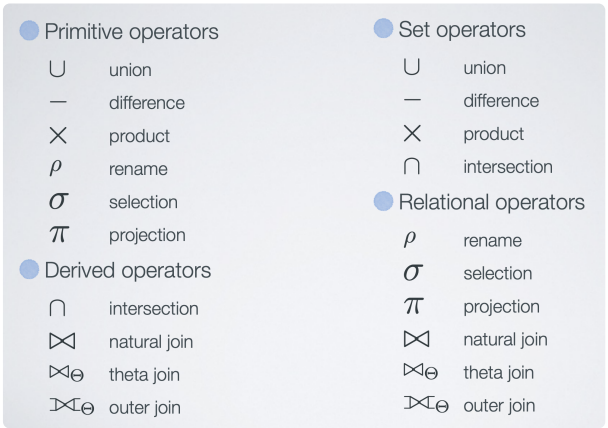
	Insertion	**Update**	**Deletion**
May violate	domain, tuple, key, referential integrity constraints	domain, tuple, key, referential integrity constraints	referential integrity constraints
Alternatives	prevent insertion or correct cause of violation	Prevent update **(restrict)** or propagate update **(cascade)**	Prevent update **(restrict)** or propagate update **(cascade)** or modify the values of referenced attributes by setting them to nULL **(set null)** or to a default value **(set default)**

Definition 24 (Relational Algebra)**.**

The relational algebra is a set of operators to manipulate whole relations: each operator processes one or more relations and produces as output a new relation

Moreover operators can be composed to create expressions, but what are these operators?

Here is a list of all the operators



Operators

Set Operators

Now we analyze the set operator by first introducing a fundamental property:

Theorem 25 (Compatibility To Union)**.**

Two relations $R_1(X_i)$ and $R_2(Y_i)$ are compatible to union if they have the same degree n and

$$\begin{cases} X_i = Y_i \\ dom(X_i) = dom(Y_i) \end{cases} \quad 1 \leq i \leq n$$

That is, *two relations are compatible to union when they have the same number of attributes and each pair of attributes has the same name and domain*. For the other set operators, the relations must satisfy the compatibility to union. That is **they must have identical structure (name and domain of columns)**

Example:

Graduated

Badge	Surname	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Manager

Badge	Surname	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Compatible To Union

Graduated

Badge	Surname	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Manager

Badge	Surname
9297	Neri
7432	Neri
9824	Verdi

Not Compatible To Union

Definition 26 (Intersection)**.**

Given two relations compatible to union, their intersection is the relation

$$R_1 \cap R_2 = \{t | t \in R_1 \cap t \in R_2\}$$

This means that the intersection contains all the tuples that are contained in both relations

$$|\cap| \leq \min\{|R_1|, |R_2|\}$$

Example:

Graduated

Badge	Surname	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Manager

Badge	Surname	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Graduated ∩ Manager

Badge	Surname	Age
7432	Neri	54
9824	Verdi	45

Intersection Example

Definition 27 (Union)**.**

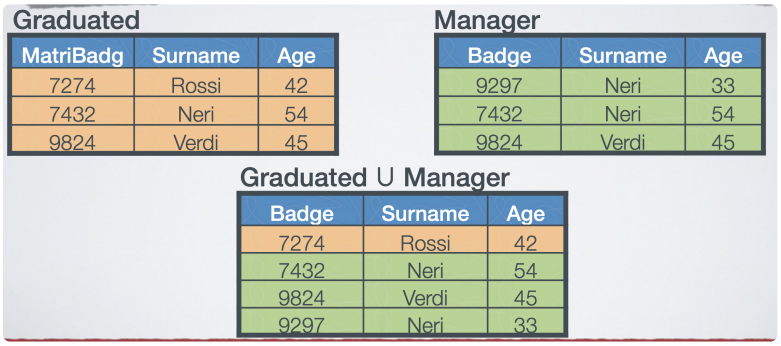
Given two relations compatible to union, their union is the relation

$$R_1 \cup R_2 = \{t | t \in R_1 \cup t \in R_2\}$$

This means that the union contains all the tuples that are present in both

$$|\cup| \geq \max\{|R_1|, |R_2|\}$$

Example



Union Example

These are set operators and therefore they operate with no order and no duplicates. In fact from the previous example we know that there are 2 identical entries in the tables. The union doesn't create a copy of these duplicates but merges them into one single entry.

Definition 28 (Difference)**.**

Given two relations compatible to union, their difference is the relation

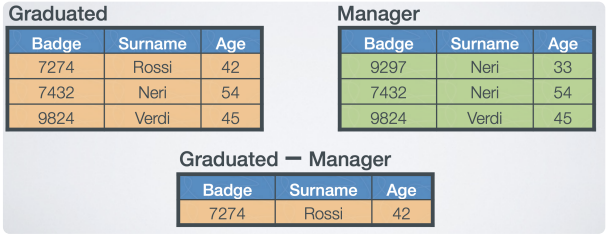
$$R_1 - R_2 = \{t | t \in R_1 \cap t \notin R_2\}$$

Notice $R_1 - R_2 \neq R_2 - R_1$

This means that we only keep the elements of R_1 that are not present in R_2.

$$|R_1 - R_2| \leq |R_1|$$

Example:



Difference Example

Definition 29 (Product (Cartesian))**.**

Given two relations **not necessarily compatible to union**, their product (cartesian) is the relation

$$R_1 \times R_2 = \{xy | x \in R_1 \cap y \in R_2\}$$

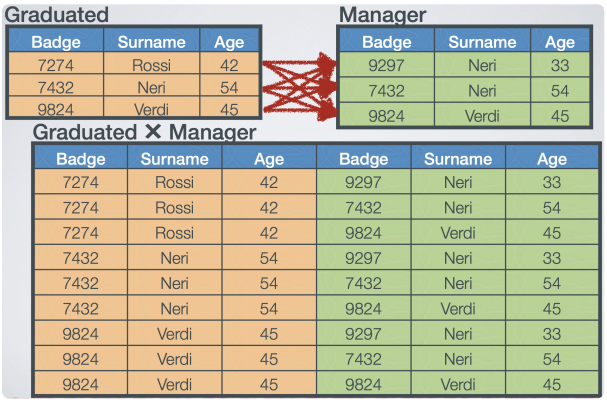
This means that the resulting table has more rows and columns, each row of one table is concatenated with each row of the other table

Let $R_1(X_1, \ldots, X_n)$ and $R_2(Y_1, .., Y_m)$ their cartesian product will have degre $q = n + m$ and be made of

$$Q = R_1 \times R_2 = (X_1, \dots, X_n, Y_1, \dots, Y_n) = (X, Y)$$

And the cardinality will be the product: $|Q| = |R_1||R_2|$ and the degree will be $\deg(Q) = \deg(R_1) + \deg(R_2)$.

Example:



Product Example

And in fact: $|Q| = 9 = 3 \cdot 3$ and $\deg(Q) = 6 = 3 + 3$.

Relational Operators

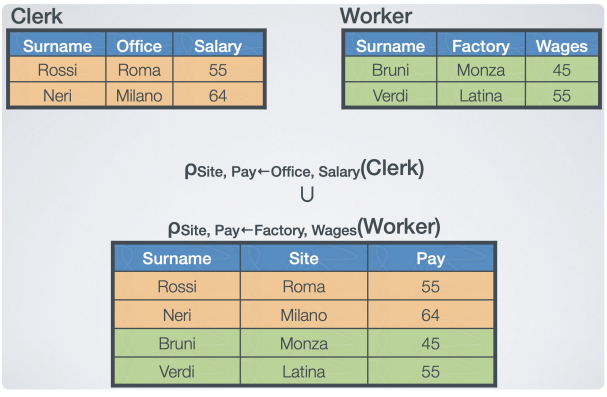
Definition 30 (Rename)**.**

Given a relation $R_1(X)$ and two sets of attributes $A \in X, B \notin X$ the rename is the relation with attributes $X - A \cup B$ such that

$$\rho_{B \leftarrow A}(R) = \{t | \exists x \in R : t[B] = x[A] \cap t[C] = x[C] \text{ if } C \neq A\}$$

This definition states that we are changing only the attribute names but not their domain or the associated values. The schema is modified but not the instance. The $t[C]$ shows that not all names have to be changed (Suppose I have Office, Salary and want to rename Salary to Wage I just have to rename $A_1 \leftarrow B_1$ not $A \leftarrow B$)

This means that we are changing the attribute (names), and only some (not all), without changing the domain or the values in the tuples.



Rename Example

Definition 31 (Selection)**.**

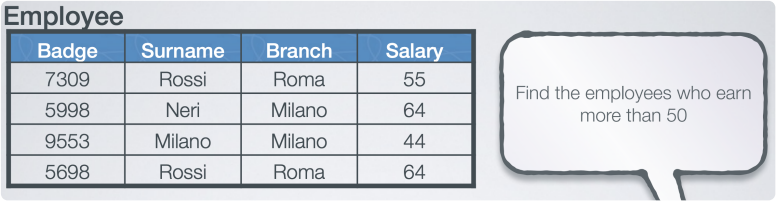
Given a relation $R(X)$ and a proposition Θ, the selection is the relation

$$\sigma_\Theta(R) = \{t | t \in R \cap \Theta\}$$

where Θ can be defined as follows:

- $X_i \theta X_j$ with X_i, X_j attributes of R and $\theta \in \{<, >, =, \neq, \leq, \geq\}$ a comparison operator
- $X_i \theta c$ with X_i attribute of R, θ a comparison operator and c a constant such that $c \in dom(X_i)$
- if ϕ, φ are propositions, then also $\phi \cap \varphi, \phi \cup \varphi \neg \varphi$ are propositions

Example:



Example

In this example R is Employee, X_i is salary, θ is $>$, then $\Theta = \text{Salary} > 50$ and finally $\sigma_{\text{salary}>50}(\text{Employee})$.

If we also want to find the employees who earn more than 50 AND work in Milan that becomes a intersection with the new proposition $\Theta' = \text{Branch} = \text{Milano}$ and thus

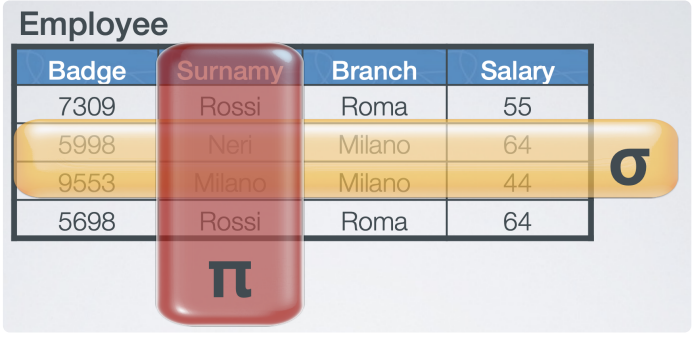
$$\sigma_{\text{Salary}>50 \cap \text{Branch}=\text{Milano}}(\text{Employee})$$

Definition 32 (Projection)**.**

Given a relation $R(X)$ and a **list** of its attributes A, the projection is the relation:

$$\pi_{A_1,\dots,A_m}(R) = \{t[A_1, \dots, A_m] | t \in R\}$$

This results in a subset of the attributes with degree m.

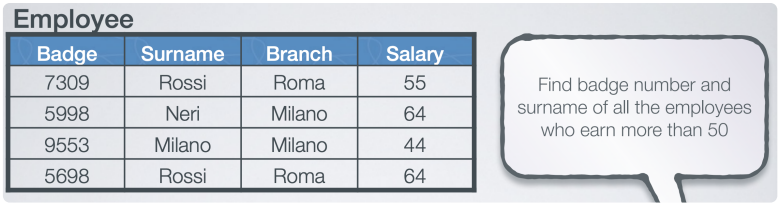


Visualization of Projection and Selection

Remark (Selection and Projection are NOT Commutative)**.**

Selection and Projection are NOT Commutative as each operation causes some information loss, in fact (usually) the selection should be applied first!

Example:



Example

In this case we must first do a selection with Salary>50 and then a projection only on Badge and Surname, that is:

$$\pi_{\text{Badgne,Surname}}(\sigma_{\text{Salary}>50}(\text{Employee}))$$

Joins

First I will write some Latex commands for some missing symbols:

$$\bowtie, \bowtie, \bowtie$$

Definition 33 (Theta-Join)**.**

Given two relations $R_1(X)$ and $R_2(Y)$ and a proposition Θ, the theta-join is the relation

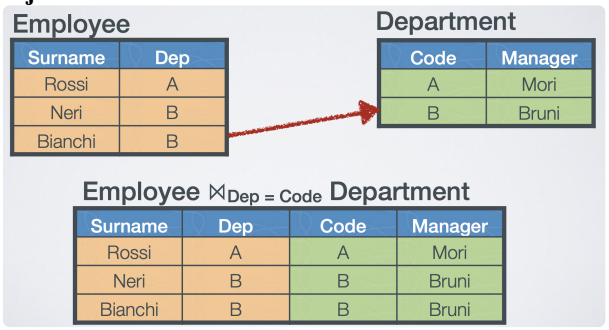
$$R_1 \bowtie_\Theta R_2 = \{t | \exists x \in R_1, y \in R_2 : x \cap y \cap \Theta\}$$

The Theta-join is an operation that combines tuples from two different relations (R_1 and R_2) into a new relation, but **only** if they satisfy a specific condition Θ.

This is equivalent (but more performant) to

$$\sigma_\Theta(R_1 \times R_2)$$

The we can also define the **equi-join** where $\Theta = =$



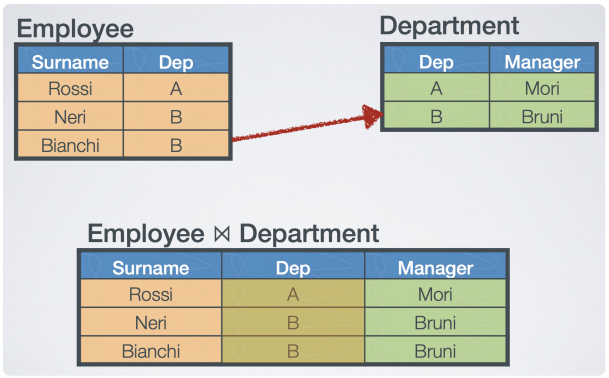
Equi Join Example

Definition 34 (Natural Join)**.**

The **natural join** is an **equi join** where the attributes of the two relations in the join condition have the **same names** and the **duplicate attributes** are **removed** from the schema of the output relation. Formally it is:
Let $R_1(XW)$ and $R_2(YZ)$ and W, Z are set of attributes with the same name, then

$$R_1 \bowtie R_2 = \pi_{X,W,Y}(R_1 \bowtie_{W_1=Z_1 \cap W_2=Z_2 \cap \ldots \cap W_m=Z_m} R_2)$$

It is **not commutative**



Natural Join Example

A quick remark on cardinality:

- In general:

$$0 \leq |R_1 \bowtie R_2| \leq |R_1||R_2|$$

- If it involves a key of R_2, then

$$0 \leq |R_1 \bowtie R_2| \leq |R_1|$$

  - If you join on an attribute that is the **key of** R_2, then **each tuple of** R_1 can match **at most one** tuple of R_2, because a key uniquely identifies a tuple.
- If the key of R_2 is a FK of R_1 the equality holds:

$$|R_1 \bowtie R_2| = |R_1|$$

Definition 35 (Full Outer Join)**.**

Given two relations $R_1(XW), R_2(YZ)$ with W, Z sets of attributes on the same domain, the **full outer join** is:

$$\begin{aligned} R_1 \overset{\bowtie}{\bowtie}_{W\theta Z} R_2 = & R_1 \bowtie_{W\sigma Z} R_2 \\ & \cup (R_1 - \pi_{X,W}(R_1 \bowtie_{W\theta Z} R_2)) \times \{Y = NULL, Z = NULL\} \\ & \cup (R_2 - \pi_{Y,Z}(R_1 \bowtie_{W\theta Z} R_2)) \times \{X = NULL, W = NULL\} \end{aligned}$$

This extends the theta-join to all the values that are excluded from the normal theta (inner) join. It also Includes the NULL value as a valid value.

Based on this we can also define the right and left outer join, keeping the union only for the left or right part, that is

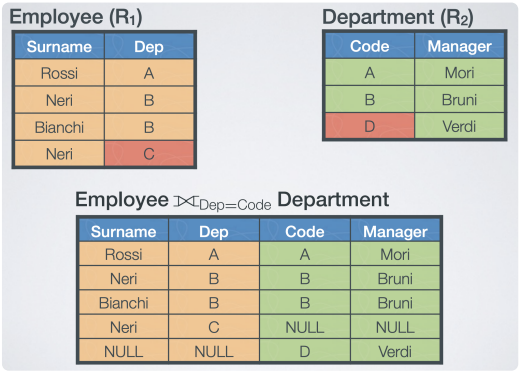
Definition 36 (Left Outer Join)**.**

$$\begin{aligned} R_1 \overset{\bowtie}{\bowtie}_{W\theta Z} R_2 = & R_1 \bowtie_{W\sigma Z} R_2 \\ & \cup (R_1 - \pi_{X,W}(R_1 \bowtie_{W\theta Z} R_2)) \times \{Y = NULL, Z = NULL\} \end{aligned}$$

Definition 37 (Right Outer Join)**.**

$$R_1 \ltimes\!\!\!\!\!\!\rightarrow_{W\theta Z} R_2 = R_1 \bowtie_{W\sigma Z} R_2 \cup (R_2 - \pi_{Y,Z}(R_1 \bowtie_{W\theta Z} R_2)) \times \{X = NULL, W = NULL\}$$

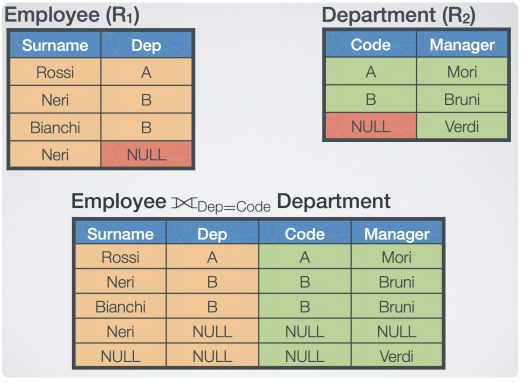
Here are some examples:
Full Outer Join



Full Outer Join Example

The first 4 lines are $R_1 \bowtie_{\text{Dep=Code}}$ while the other are added via the double NULL union.
The first excluded line is a result of the left outer join (only left values are not NULL)
The last line is the right outer join

Here is an example with NULL values in the OG tables



Example

Remark.

A joint is more efficient than set operations, so use set operations only if the query cannot be achieved with a joint

Remark (NULL Values are not included)**.**

NULL values are not included in the operators.
In fact, suppose to have a relation $R(A)$ where there is a NULL value associated to an attribute A_i where $\text{dom}(A_i) \in \mathbb{N}$ in a tuple t_j, then

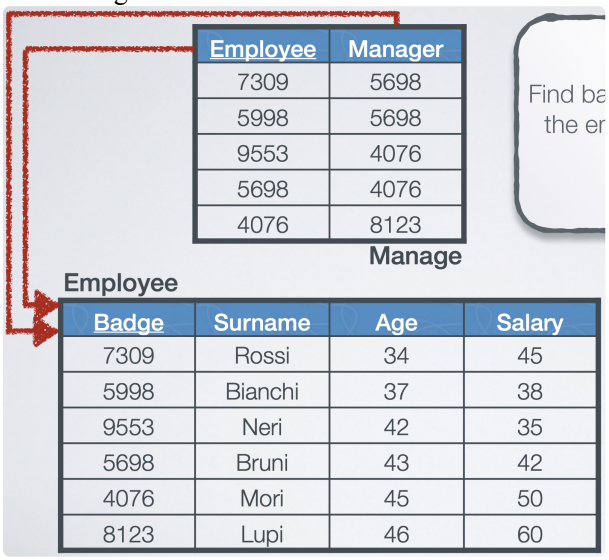
$$R(A) \neq \begin{cases} \sigma_{A_i \geq 0}(R) \\ \sigma_{A_i < 0}(R) \end{cases} = R(A) - \sigma_{A_i \text{ IS NULL}}(R) = R_1 - t_j$$

And to return the whole table we must include a statement

$$R(A) = \begin{cases} \sigma_{A_i \geq 0}(R) \\ \sigma_{A_i < 0}(R) \\ \sigma_{A_i \text{ IS NULL}}(R) \end{cases}$$

Exercises

For the exercise we will use the following tables.



Manage

Employee	Manager
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Employee

Badge	Surname	Age	Salary
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Tables

1. Find badge, surname, age and salary of the employees who earn more than 42
   This is a simple selection on employee and operation salary > 42:

   $$\sigma_{\text{Salary}>42}(\text{Employee})$$

2. Find badge, surname, and age of the employees who earn more than 42
   This is as in 1) but with an added projection on the specified attributes:

   $$\pi_{\text{Badge, Surname, Age}}(\sigma_{\text{Salary}>42}(\text{Employee}))$$

3. Find the badge number of the managers whose employees earn more than 42
   Here we must first find 1) and then do a join on Badge=Employee, then project on manager:

   $$\pi_{\text{Manager}}(\text{Manage} \bowtie_{\text{Employee}=\text{Badge}} \sigma_{\text{Salary}>42}(\text{Employee}))$$

   The join will produce the following table where Employee=Badge

Employee (=Badge)	Manager	Badge (=Employee)	Surname	Age	Salary

Semantically this table contains the information on the employees with their managers

4. Find the surname and the salary of the managers whose employees earn more than 42
   This is similar to before, just that before doing the projection we must join again on Badge=Manager

   $$\pi_{\text{M.Sur,M.Sal}}(\rho_{M.A_i \leftarrow A_i}(\text{Employee}) \bowtie_{\text{M.Bdg}=\text{Mng}} \text{Manage} \bowtie_{\text{Emp}=\text{Bdg}} \sigma_{\text{Sal}>42}(\text{Employee}))$$

   This table will be (the leftmost was renamed to $\rho_{M.A_i \leftarrow A_i}(\text{Employee})$)

M.Bdg (=Mng)	M.Surn	M.Age	M.Sal	Emp (=Bdg)	Mng (=M.Bdg)	Bdg (=Emp)	Sur	Age	Sal

This table will have the M. part dedicated to the info of the manager and the right side to the employee with this manager.

5. Find the surname and the salary of the managers who earn more than 45
   First select, then join on Badge=Employee and then project

$$\pi_{\text{Surname,Salary}}(\text{Manage} \bowtie_{\text{Manager=Badge}} \sigma_{\text{Salary}>45}(\text{Employee}))$$

   In fact this table has the infos of the managers

Employee	Manager (=Badge)	Badge (=Manager)	Surname	Age	Salary

6. Find the employees who earn more than their managers, returning badge, surname and salary of both the employee and the manager
   First we create the table of 4), then we select the Salary > M.salary and then we project

$$\pi_{\text{M.Sur,M.Sal}}(\sigma_{\text{Sal}>\text{M.Sal}}((\rho_{M.A_i \leftarrow A_i}(\text{Employee}) \bowtie_{\text{M.Bdg=Mng}} \text{Manage} \bowtie_{\text{Emp=Bdg}} \text{Employee}))$$

7. Find the badge number of the managers whose employees ALL earn more than 42
   We take all Managers (projection) and subtract all managers who have at least one with less salary

$$\pi_{\text{Manager}}(\text{Manage}) - \pi_{\text{Manager}}(\text{Manager} \bowtie_{\text{Employee=Badge}} \sigma_{\text{Salary}\leq 42}(\text{Employee}))$$

> **Remark.**
>
> The relational algebra expressions are read from and to beginning (right to left, bottom to top)

Given a DB schema and instance, we can perform 2 operations: **update, query**

5.2) Update Operation

An update is a function that maps the given DB instance into a new DB instance which is valid wrt all the integrity constraints

In fact an **insertion or update** might violate domain, tuple, key, referential integrities.

- **insertion:** We can either prevent the insertion or try to correct the constraint violaton.
- **update:** prevent (restrict) or propagate (cascade) the update
- **deletion:** this might violate referential constraints, we can either **restrict, cascade or** set the referenced attributes to a set value or NULL.

6) SQL

This chapter consists in introducing SQL and the direct application of relational algebra.

Definition 38 (Structured Query Language (SQL))**.**

Structured Query Language (SQL) is the standard relational language to work with database management systems (DMS)

It is a declarative language and consists of

- Data Definition Languaga (DDL)
- Data Manipulation Language (DML)

It implements the relational model, relational algebra and some extensions

Catalog Vs Schema

First we must distinguish a catalog (database) from the schema (tables, domains, etc...)

Feature	Catalog	Schema
Level	Top-level container	Namespace inside a catalog
Contains	Schemas	Tables, Views, Functions, etc.
Purpose	Separates databases	Organizes objects within a DB
Typical name	Database name (SalesDB, HRDB)	Usually “public”, “sales”, etc.

Tables And Queries

Definition 39 (Table)**.**

A table is a collection of zero or more rows zero or more and columns.

- Rows are not ordered
- Each row has a type called **row type** which is the same for all the rows in the table

The **Base Table** contains the SQL data and cannot have columns with the same name

A SQL table is a **multi set** of tuples, it can contain duplicate elements. In a strict sense it therefore is not a relation

Definition 40 (Query)**.**

A query is an operation concerning one or more base tables and returning a derived table

- The rows can be ordered
- More than one column with the same name

Tokens, Keywords and Identifiers

Tokens are lexical units of the language. They can be used as identifiers and keywrds (case INsensitive) and as literal

Keywords are reserved words that have a special meaning in SQL. They are part of the syntax and cannot be used as identifiers unless quoted (" · ").

SQL tokens are case INsensitive, however tables and identifiers are case sensitive.

There are 2 types of identifiers:

- regular: 128 char string. The first must be a letter, then any combination of letters, number and underscores. No reserved keywords
- delimited: string between quotes. Can also be a reserved keyword (SELECT is reserved, "SELECT" is valid identifier)

6.2) Data Definition Language (DDL)

DDL vs DML (Data Manipulation Language)
DDL allows to create database + tables
DML allows to extract , analyze and manipulate data

The creation of a catalog is implementation-defined. In PostgreSQL we have

```
CREATE DATABASE <dB-name> [owner <username>] [ENCODING <encoding-name>]
```

[ ] = optional field
Encoding can be set to 'UTF-8' to support italian language

to delete a catalog just use:

```
DROP DATABASE <dB-name>
```

Every SQL statement ends with a semicolon

The catalog by default comes with a "public" schema. To create a custom schema use:

```
CREATE SCHEMA <schema-name> [AUTHORIZATION <username>]
```

To drop a schema there are two options:

```
DROP SCHEMA <schema-name> [CASCADE | RESTRICT]
```

- Restrict: default behaviour, deletes only if empty
- Cascade: deletes all objects in schema and then drops the schema

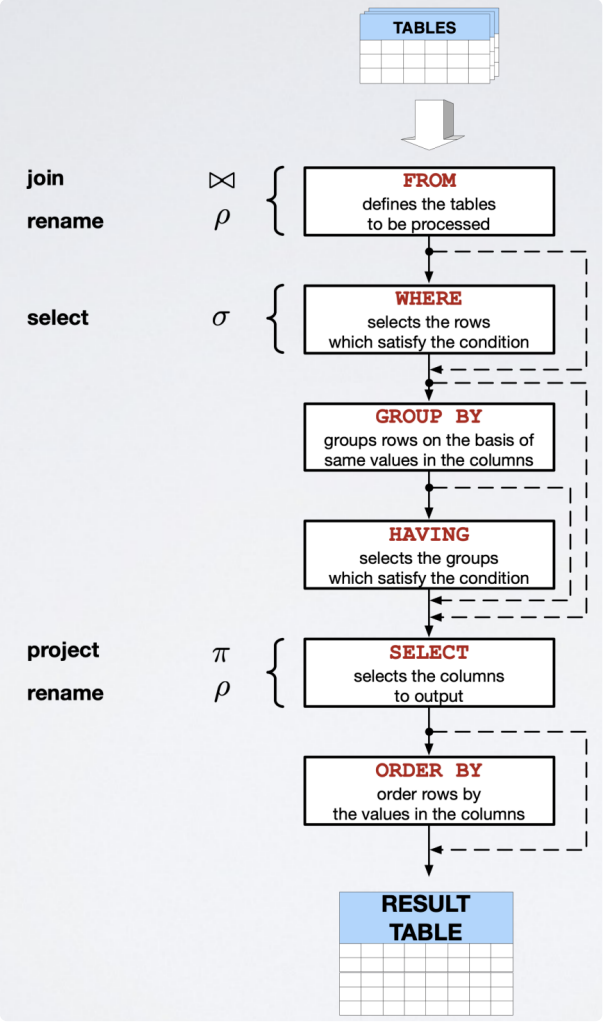
Data Type: domain of data in column, and it **also represents the types of operations we can do**.

6.3) Data Manipulation Language (DML)

SELECT in SQL = π (projection)
This is since you select a column list, so it is actualy a projection!

```
SELECT [DISTINCT | ALL] <column-list> | *
    FROM <table-name>
        [<join-type> JOIN <table-name> ON <join-condition>]
    [WHERE <expression>]
    [GROUP BY <column-name> [HAVING <expression>]]
    [ORDER BY <column-name> [ASC | DESC]]
        [ ,<column-name> [ASC | DESC]]
```

ALL $\rightarrow$ Bag semantics (allows for duplicates)
DISTINCT $\rightarrow$ Set semantics
$*$ $\rightarrow$ selects full table



Recap

To concatenate we use:

```
|| ' ' ||
```

7) Exam Example

Given the following database,

Airport

ID	Name	City
CIA	Ciampino	Rome
FCO	Fiumicino	Rome
LIN	Linate	Milan
MXP	Malpensa	Milan
VCE	Marco Polo	Venice
CDG	Charles de Gaulle	Paris
ORY	Orly	Paris
AMS	Schiphol Airport	Amsterdam
IAD	Dulles Intl	Washington
DCA	Ronald Reagan National	Washington
LAX	International	Los Angeles
DXB	International	Dubai
MEL	Melbourne Airport	Melbourne
NRT	Narita Intl	Tokyo
HND	Tokyo Intl (Haneda)	Tokyo

City

Name	Country
Rome	Italy
Milan	Italy
Venice	Italy
Paris	France
Amsterdam	The Netherlands
Washington	USA
Los Angeles	USA
Dubai	United Arab Emirates
Melbourne	Australia
Tokyo	Japan

TravelPlan

ID	Departure	Arrival	Created	Customer
852e3224-6072-4701-8e74-ecab5d6bd35c	Venice	Rome	2019-06-19 06:26:28.71+02	Derek Shepherd
9c4c8629-4c72-404a-8eb8-40b532bf3c9b	Venice	Tokyo	2019-06-12 09:56:18.82+02	Mark Sloan
d11ba110-2fb2-4cd5-8021-bbb367c3bd2b	Milan	Los Angeles	2019-05-25 19:12:38.12+02	Meredith Grey
c739e02c-00b8-4dff-9068-8bd5bc496a39	Rome	Los Angeles	2019-07-12 09:25:11.99+02	Mark Sloan
9d4bb981-6bbc-430f-9ba8-499c501071c5	Rome	Los Angeles	2019-04-22 16:42:52.11+02	Alex Karev
902e863c-eada-4bcc-8f17-cfd71d6a7a4b	Milan	Tokyo	2019-02-12 06:46:52.11+02	Alex Karev
4ef6e952-4c51-40ba-a4af-142b84c566de	Milan	Tokyo	2019-03-21 16:21:25.09+02	Cristina Yang
909febe9-e146-4c2a-80e7-18d29f3e7729	Milan	Tokyo	2019-08-11 12:11:15.52+02	Cristina Yang
4a3ace7c-7f88-44d0-95ef-d433ed122a73	Milan	Washington	2019-01-01 02:24:19.10+02	Jackson Avery
d24de4db-3c94-459b-9269-3de56739ec81	Milan	Melbourne	2019-05-21 13:42:45.97+02	Arizona Robbins
df517c38-7a9f-4120-84b3-01b8c8de1358	Venice	Paris	2019-07-23 11:19:51.73+02	Callie Torres
af384c70-1895-4b03-8ae9-b70c4795e914	Amsterdam	Los Angeles	2019-06-20 09:00:00.01+02	Addison Montgomery

Belong

TravelPlan	Flight	DepartureDate	Leg
852e3224-6072-4701-8e74-ecab5d6bd35c	AZ1466	2019-09-21	1
9c4c8629-4c72-404a-8eb8-40b532bf3c9b	KL1656	2019-06-15	1
9c4c8629-4c72-404a-8eb8-40b532bf3c9b	KL863	2019-06-16	2
d11ba110-2fb2-4cd5-8021-bbb367c3bd2b	AZ2015	2019-07-01	1
d11ba110-2fb2-4cd5-8021-bbb367c3bd2b	AZ620	2019-07-01	2
c739e02c-00b8-4dff-9068-8bd5bc496a39	AZ620	2019-07-17	1
9d4bb981-6bbc-430f-9ba8-499c501071c5	EK98	2019-05-10	1
9d4bb981-6bbc-430f-9ba8-499c501071c5	EK408	2019-05-11	2
9d4bb981-6bbc-430f-9ba8-499c501071c5	QF95	2019-05-11	3
902e863c-eada-4bcc-8f17-cfd71d6a7a4b	AZ786	2019-02-20	1
4ef6e952-4c51-40ba-a4af-142b84c566de	EK102	2019-04-19	1
4ef6e952-4c51-40ba-a4af-142b84c566de	EK318	2019-04-20	2
909febe9-e146-4c2a-80e7-18d29f3e7729	EK92	2019-10-23	1
909febe9-e146-4c2a-80e7-18d29f3e7729	EK312	2019-10-24	2
4a3ace7c-7f88-44d0-95ef-d433ed122a73	KL1620	2019-02-14	1
4a3ace7c-7f88-44d0-95ef-d433ed122a73	KL651	2019-02-14	2
d24de4db-3c94-459b-9269-3de56739ec81	KL1620	2019-06-24	1
d24de4db-3c94-459b-9269-3de56739ec81	KL427	2019-06-24	2
d24de4db-3c94-459b-9269-3de56739ec81	EK408	2019-06-25	3

Flight

ID	Departure	Arrival	DepartureTime	Duration
KL1628	MXP	AMS	06:40+02:00	01:55
EK102	MXP	DXB	11:20+02:00	06:00
EK92	MXP	DXB	22:20+02:00	06:25
AZ786	MXP	NRT	15:25+02:00	12:10
AF1331	MXP	CDG	13:55+02:00	01:30
AZ2015	LIN	FCO	07:15+02:00	01:10
AZ2025	LIN	FCO	08:20+02:00	01:10
AZ2037	LIN	FCO	12:00+02:00	01:10
KL1620	LIN	AMS	10:50+02:00	01:55
AF1213	LIN	CDG	09:50+02:00	01:35
KL1656	VCE	AMS	16:55+02:00	02:00
AZ1466	VCE	FCO	11:20+02:00	01:10
AF1727	VCE	CDG	15:20+02:00	01:50
AZ620	FCO	LAX	09:15+02:00	13:05
UA43	FCO	IAD	09:45+02:00	09:45
EK98	FCO	DXB	15:25+02:00	06:00
KL863	AMS	NRT	17:50+02:00	11:10
KL651	AMS	IAD	13:20+02:00	08:20
DL79	AMS	LAX	15:15+02:00	11:47
KL427	AMS	DXB	14:30+02:00	06:35
EK205	DXB	MXP	09:45+02:00	06:35
EK408	DXB	MEL	02:40+02:00	13:10
EK406	DXB	MEL	10:15+02:00	13:20
EK215	DXB	LAX	08:55+02:00	16:00
EK318	DXB	NRT	02:40+02:00	09:55
EK312	DXB	HND	08:00+02:00	09:45
QF95	MEL	LAX	20:55+02:00	14:20
QF79	MEL	NRT	09:10+02:00	10:25
EK407	MEL	DXB	21:15+02:00	14:00

Pasted image 20251217104227.png

list the travel plans (id, departure, arrival, customer) to which there are no associated flights yet using relational algebra.

Select one:

- $\pi_{id,departure,customer}($ $\sigma_{flight\ IS\ NULL}$ ($TravelPlan \bowtie_{id=travelPlan}\ Belong$))
- $\pi_{id,departure,arrival,customer}($ $\sigma_{flight\ IS\ NULL}$ ($TravelPlan \bowtie_{id=travelPlan}\ Belong$))
- $\pi_{id,departure,arrival,customer}($ $\sigma_{flight\ IS\ NULL}$ ($TravelPlan \bowtie_{id=travelPlan} Belong$)) ✓
- $\pi_{id,departure,arrival,customer}($ $\sigma_{flight\ IS\ NOT\ NULL}$ ($TravelPlan \bowtie_{id=travelPlan} Belong$))

Risposta corretta.

The correct answer is:

$\pi_{id,departure,arrival,customer}($ $\sigma_{flight\ IS\ NULL}$ ($TravelPlan \bowtie_{id=travelPlan} Belong$))

Pasted image 20251217110128.png

State the definition of **attribute** in the ER model

Select one:

- An attribute in the ER model is a local property of an entity or relationship, relevant for the application. ✓
- An attribute in the ER model is a local property of an entity, relevant for the application.
- An attribute in the ER model is a local property mapping each instance to several different values belonging to a set called domain.
- An attribute in the ER model is a rule expressed on the schema, which specifies a condition that must be met for each instance of the schema.

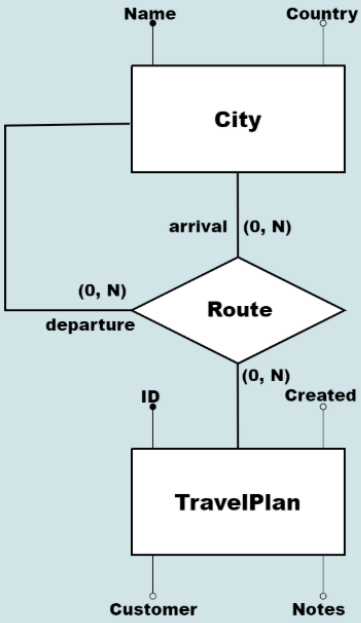
Risposta corretta.

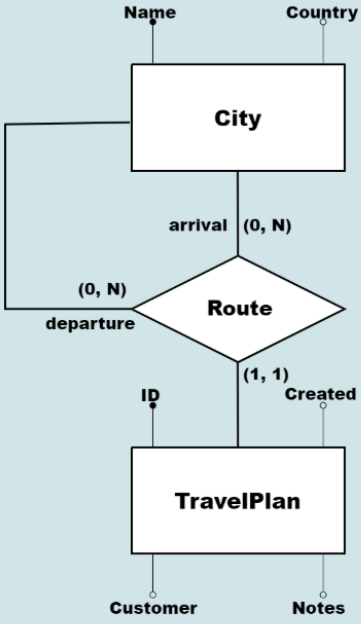
The correct answer is: An attribute in the ER model is a local property of an entity or relationship, relevant for the application.

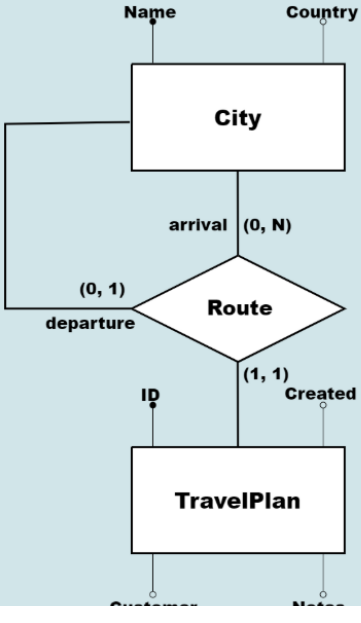
Pasted image 20251217110137.png

Choose the correct ER representation of the relationship between City and TravelPlan.

Select one:

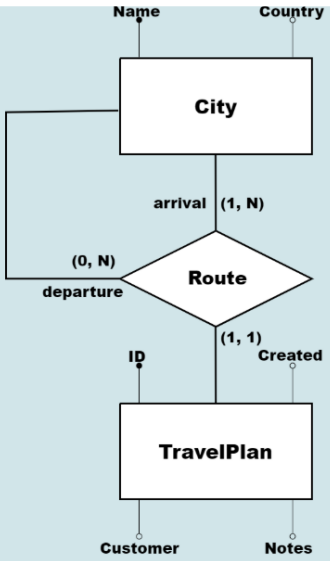






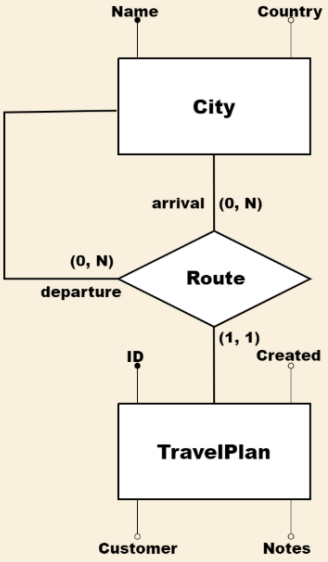
Customer Notes

Pasted image 20251217110201.png



Risposta errata.

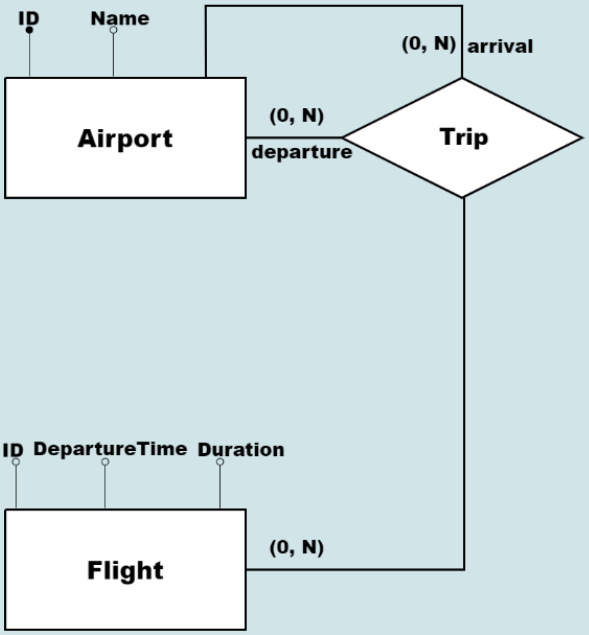
The correct answer is:

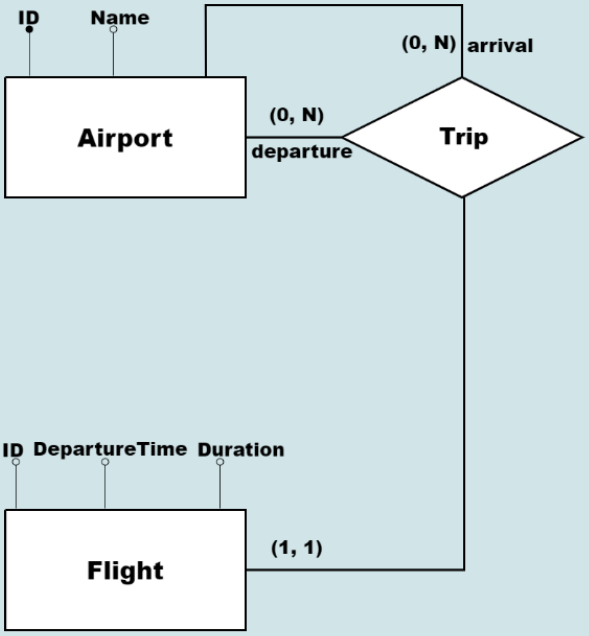


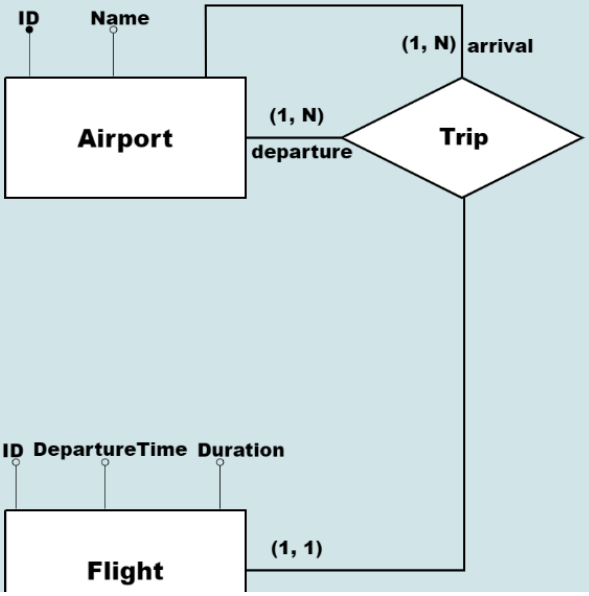
Pasted image 20251217110209.png

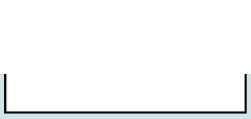
Choose the correct ER representation of the relationship between Airport and Flight.

Select one:

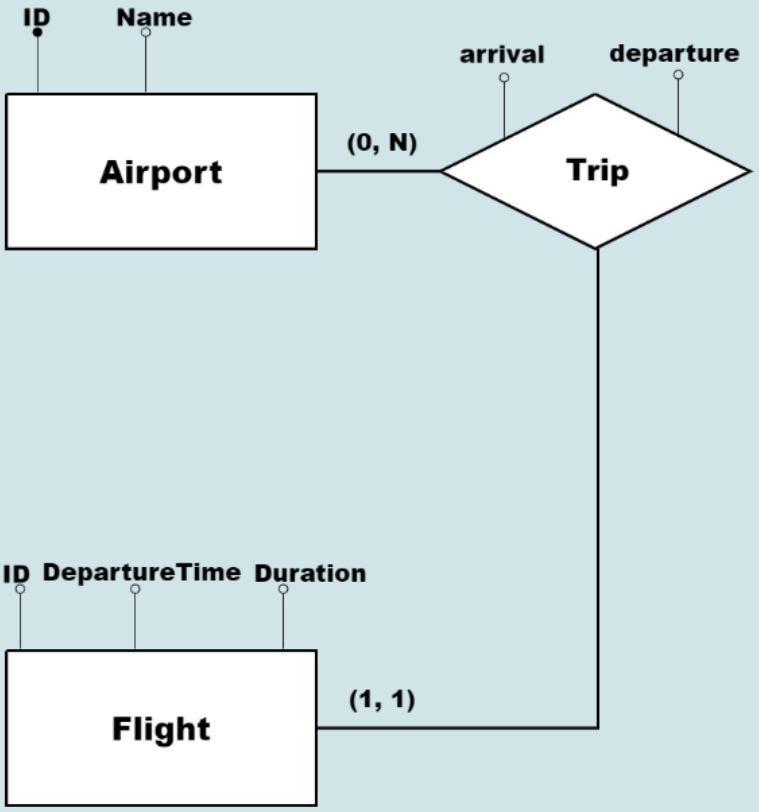




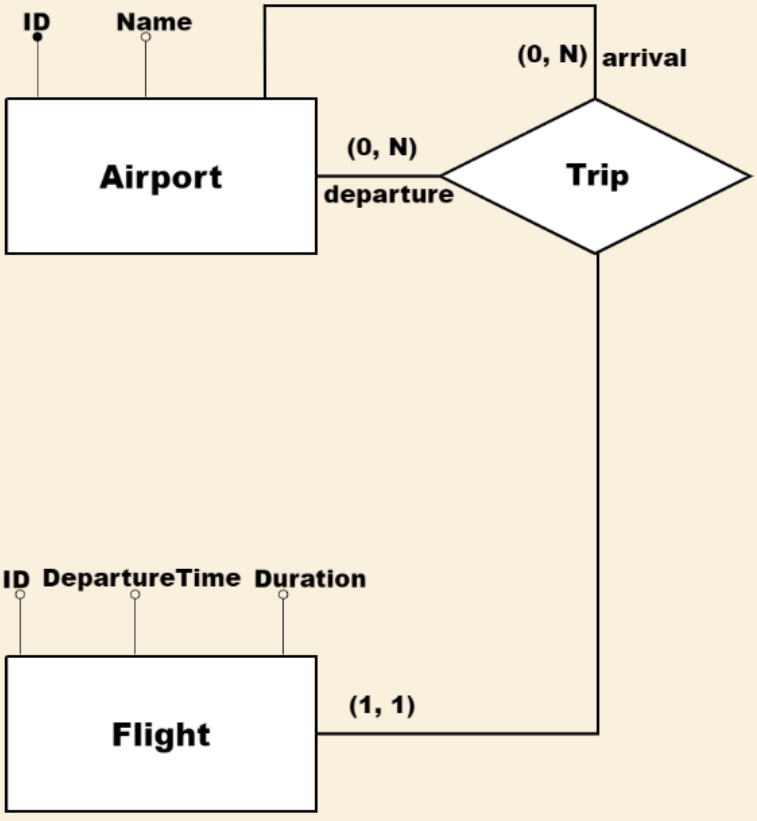




Pasted image 20251217110234.png



Risposta corretta.



The correct answer is:

Pasted image 20251217110243.png

Choose the correct SQL statement to create the Flight table.

Select one:

- CREATE TABLE Flight (
  id VARCHAR(6),
  departure AirportCode NOT NULL,
  arrival AirportCode NOT NULL,
  departureTime TIME WITH TIME ZONE,
  duration INTERVAL HOUR TO MINUTE NOT NULL,
  PRIMARY KEY (id),
  FOREIGN KEY (departure) REFERENCES Airport(id),
  FOREIGN KEY (arrival) REFERENCES Airport(id)
  );

- CREATE TABLE Flight (
  id VARCHAR(6),
  departure AirportCode NOT NULL,
  arrival AirportCode NOT NULL,
  departureTime TIMESTAMP WITH TIME ZONE,
  duration INTERVAL HOUR TO MINUTE NOT NULL,
  PRIMARY KEY (id),
  FOREIGN KEY (id) REFERENCES Belong(flight),
  FOREIGN KEY (departure) REFERENCES Airport(id),
  FOREIGN KEY (arrival) REFERENCES Airport(id)
  );

- CREATE TABLE Flight (
  id VARCHAR(6),
  departure AirportCode NOT NULL,
  arrival AirportCode NOT NULL,
  departureTime TIME WITH TIME ZONE,
  duration INTERVAL HOUR TO MINUTE NOT NULL,
  PRIMARY KEY (id),
  FOREIGN KEY (id) REFERENCES Belong(flight)
  );

```
CREATE TABLE Flight (
    id VARCHAR(6),
    departure AirportCode NOT NULL,
    arrival AirportCode NOT NULL,
    departureTime TIME WITH TIME ZONE,
    duration INTERVAL HOUR TO MINUTE NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (id) REFERENCES Belong(flight),
    FOREIGN KEY (departure) REFERENCES Airport(id),
    FOREIGN KEY (arrival) REFERENCES Airport(id)
);
```

Pasted image 20251217110257.png

Risposta corretta.

The correct answer is: CREATE TABLE Flight (
id VARCHAR(6),
departure AirportCode NOT NULL,
arrival AirportCode NOT NULL,
departureTime TIME WITH TIME ZONE,
duration INTERVAL HOUR TO MINUTE NOT NULL,
PRIMARY KEY (id),
FOREIGN KEY (departure) REFERENCES Airport(id),
FOREIGN KEY (arrival) REFERENCES Airport(id)
);

Pasted image 20251217110304.png

Choose the correct SQL statement to create the TravelPlan table.

Time left

Select one:

- CREATE TABLE TravelPlan (
  id TEXT PRIMARY KEY,
  departure CityName NOT NULL,
  arrival CityName NOT NULL,
  created TIMESTAMP NOT NULL,
  customer VARCHAR(128) NOT NULL,
  notes VARCHAR(128),
  FOREIGN KEY (arrival) REFERENCES City(name)
  );

- CREATE TABLE TravelPlan (
  id UUID,
  departure CityName NOT NULL,
  arrival CityName NOT NULL,
  created TIMESTAMP WITH TIME ZONE NOT NULL,
  customer VARCHAR(128) NOT NULL,
  notes VARCHAR(128),
  PRIMARY KEY (id)
  );

- CREATE TABLE TravelPlan (
  id TEXT,
  departure CityName NOT NULL,
  arrival CityName NOT NULL,
  created TIMESTAMP NOT NULL,
  customer VARCHAR(128) NOT NULL,
  notes VARCHAR(128),
  PRIMARY KEY (id),
  FOREIGN KEY (departure) REFERENCES City(name),
  FOREIGN KEY (arrival) REFERENCES City(name)
  );

- CREATE TABLE TravelPlan (
  id UUID,
  departure CityName NOT NULL,
  arrival CityName NOT NULL,
  created TIMESTAMP WITH TIME ZONE NOT NULL,
  customer VARCHAR(128) NOT NULL,
  notes VARCHAR(128),
  PRIMARY KEY (id),

```
    FOREIGN KEY (departure) REFERENCES City(name),
    FOREIGN KEY (arrival) REFERENCES City(name)
);
```

Pasted image 20251217110319.png

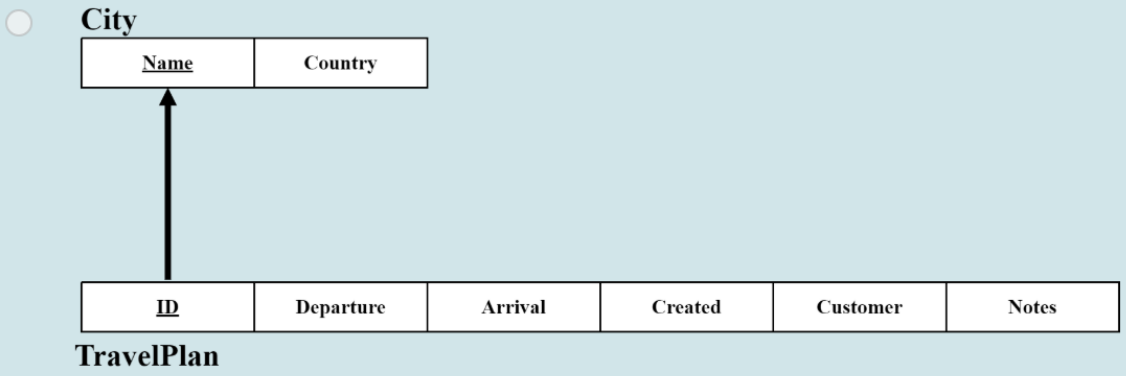
Risposta corretta.

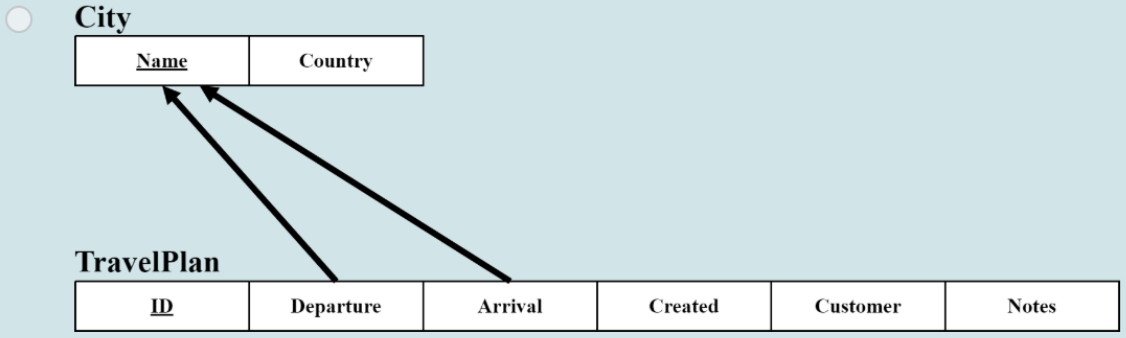
The correct answer is: CREATE TABLE TravelPlan (
id UUID,
departure CityName NOT NULL,
arrival CityName NOT NULL,
created TIMESTAMP WITH TIME ZONE NOT NULL,
customer VARCHAR(128) NOT NULL,
notes VARCHAR(128),
PRIMARY KEY (id),
FOREIGN KEY (departure) REFERENCES City(name),
FOREIGN KEY (arrival) REFERENCES City(name)
);

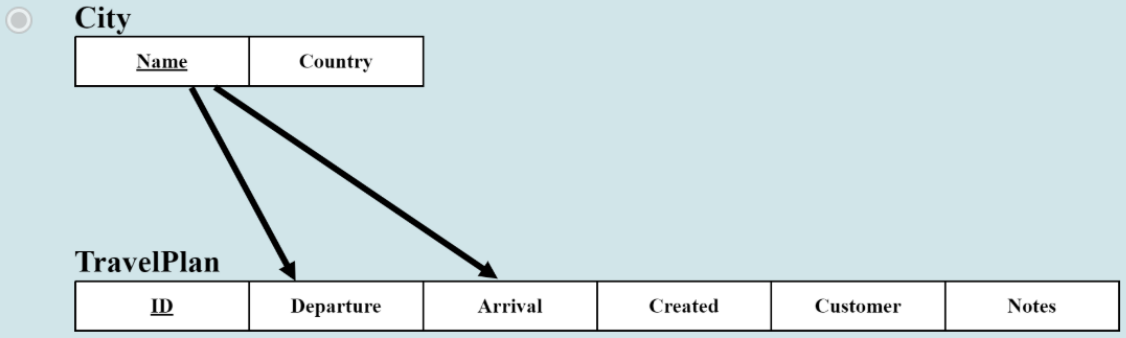
Pasted image 20251217110325.png

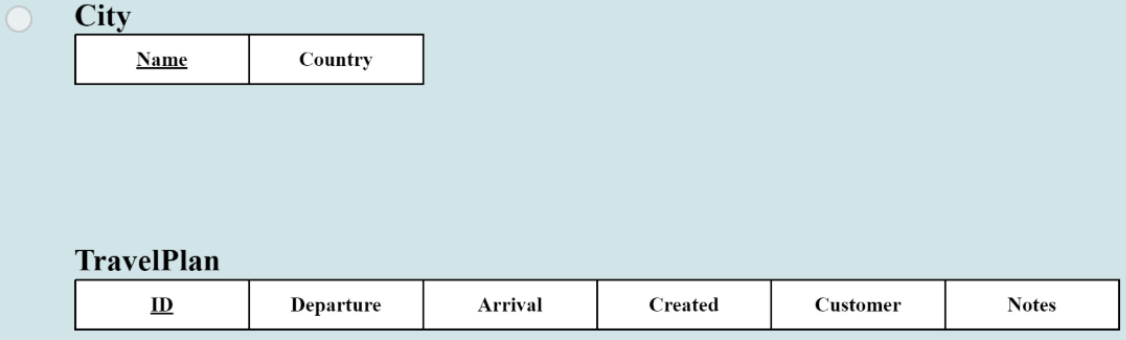
Choose the correct Relational representation for TravelPlan and City tables.

Select one:





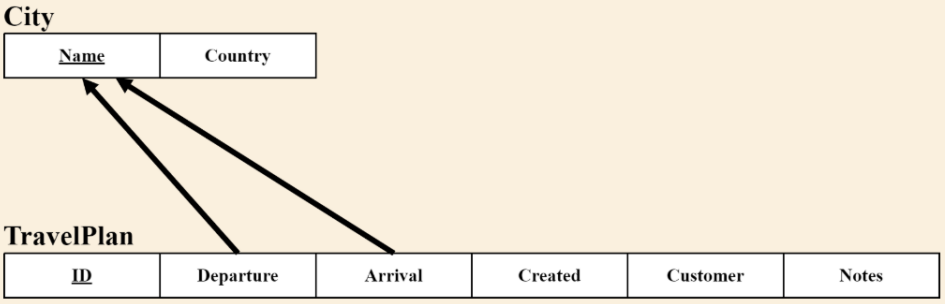




Pasted image 20251217110053.png

Risposta errata.

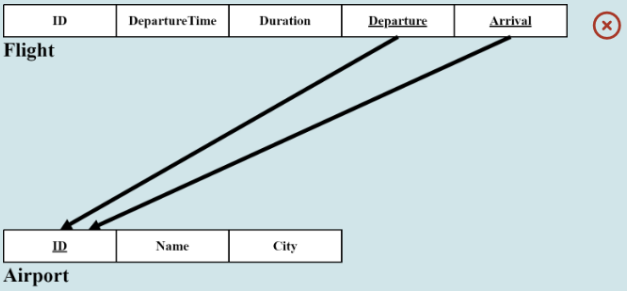
The correct answer is:



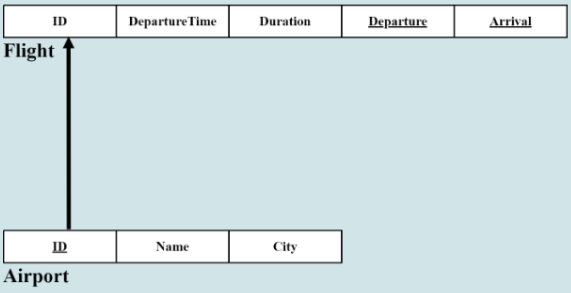
Pasted image 20251217110101.png

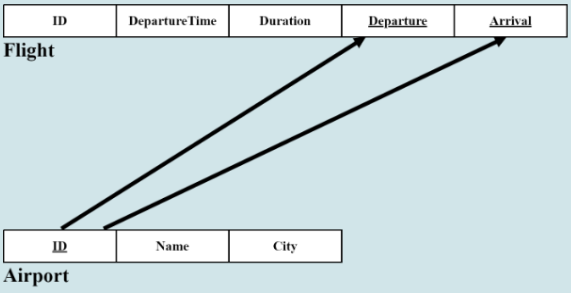
Choose the correct Relational representation for Flight and Airport tables.

Select one:



None of the other choices.





Risposta errata.

The correct answer is: None of the other choices.

Pasted image 20251217110007.png

Using SQL, display the itinerary of the travel from Rome to Los Angeles of the customer called Alex Karev. For each leg of the travel, in ascending order of leg, display the departure and arrival airports, and the departure date.

Select one:

```
SELECT F.departure, F.arrival, B.departureDate
 FROM TravelPlan AS TP INNER JOIN BELONG AS B ON TP.id = B.travelPlan INNER JOIN Flight AS F ON B.flight =
 WHERE TP.Customer = `Alex Karev' AND TP.departure = `Rome' OR TP.arrival = `Los Angeles'
 ORDER BY leg ASC;
```

```
SELECT F.departure, F.arrival, B.departureDate
 FROM TravelPlan AS TP INNER JOIN BELONG AS B ON TP.id = B.travelPlan INNER JOIN Flight AS F ON B.flight =
 WHERE TP.Customer = `Alex Karev' AND TP.departure = `Rome' AND TP.arrival = `Los Angeles'
 ORDER BY leg ASC;
```

None of the above answers ⊗

```
SELECT F.departure, F.arrival, B.departureDate
 FROM TravelPlan AS TP LEFT JOIN BELONG AS B ON TP.id = B.travelPlan INNER JOIN Flight AS F ON B.flight =
 WHERE TP.Customer = `Alex Karev' AND TP.departure = `Rome' AND TP.arrival = `Los Angeles'
 ORDER BY leg DESC;
```

Risposta errata.

The correct answer is:

```
SELECT F.departure, F.arrival, B.departureDate
 FROM TravelPlan AS TP INNER JOIN BELONG AS B ON TP.id = B.travelPlan INNER JOIN Flight AS F ON B.flight = F.id
 WHERE TP.Customer = `Alex Karev' AND TP.departure = `Rome' AND TP.arrival = `Los Angeles'
 ORDER BY leg ASC;
```

Pasted image 20251217111032.png

Select one:

- ```
  SELECT id, departure, arrival, customer
   FROM TravelPlan AS TP INNER JOIN Belong AS B on TP.id = B.travelPlan
   WHERE flight IS NULL;
  ```
- ```
  SELECT id, departure, arrival
   FROM TravelPlan AS TP FULL OUTER JOIN Belong AS B on TP.id = B.travelPl
   WHERE flight IS NULL;
  ```
- ```
  SELECT id, departure, arrival, customer
   FROM TravelPlan AS TP FULL OUTER JOIN Belong AS B on TP.id = B.travelPl
   WHERE flight IS NOT NULL;
  ```
  ⊗
- ```
  SELECT id, departure, arrival, customer
   FROM TravelPlan AS TP LEFT JOIN Belong AS B on TP.id = B.travelPlan
   WHERE flight IS NULL;
  ```

Risposta errata.

The correct answer is:

```
SELECT id, departure, arrival, customer
 FROM TravelPlan AS TP LEFT JOIN Belong AS B on TP.id = B.travelPlan
 WHERE flight IS NULL;
```

Pasted image 20251217111401.png

Among the different meanings of the NULL value, choose the wrong one.

Select one:

- Undefined value
- Not available value
- Unknown value
- False value ✓

Risposta corretta.

The correct answer is: False value

Given the following definition, choose the proper **quality of a schema.**

"The schema has to represent all the concepts and properties relevant to the mini-world and identified in the requirements"

Select one:

- Understandability
- Completeness ✓
- Correctness
- Minimality

Risposta corretta.

The correct answer is: Completeness

Pasted image 20251217105939.png